

MAE 6291

Internet of Things for Engineers

Prof. Kartik Bulusu, MAE Dept.

Week 12 [04/15/2026]

- Localhost
- Tunnel – Exposing your localhost
- In-class: run the Flask API code and create a localhost
- Install Cloudflare localhost tunnelling software
- Expose localhost to the internet by a tunnel

`git clone https://github.com/gwu-mae6291-iot/spring2026_codes.git`



School of Engineering
& Applied Science

Spring 2026

THE GEORGE WASHINGTON UNIVERSITY

Photo: Kartik Bulusu

Final projects [Things to remember]





Reminders on student deliverables:

1. [Upload by 11:00 AM on April 21, 2026] Final project presentation [25 points]
2. [Upload by 11:00 AM on April 21, 2026] Final project 2-minute video [25 points]
3. [April 22, 2026] Final project demo [Tied to #1]
4. [Upload by April 21, 2026] Executive summary [10 points]
5. [Upload on April 26, 2026] Final project written report [25 points]
6. [Upload on April 29, 2026] Up to 2 Journal paper reviews by Grad students [10 points]
7. [On Google form] Share the Github repo of midterm and final projects [Tied to #1]
8. Attendance is mandatory, no exceptions and no extensions



Expose the localhost



Understanding 127.0.0.1: The localhost Address

The IP address [127.0.0.1](https://www.iana.org/reserved) is commonly known as **localhost**.

- It is a special IP address used by a computer to refer to itself.
- Instead of connecting to an external network or the internet, this address is used for loopback communication within the same machine.

Network Configuration:

A server or program that you are running locally will frequently give you instructions on how to access it via **localhost**, along with the necessary ports and extra paths.

Sources:
Medium: <https://medium.com/devgorilla/exploring-localhost-and-127-0-0-1-what-is-the-difference-c109f7f29>
localhost: <https://todaybestreports.com/127-0-0-1-162893-localhost-port-usage-explain/>
IP address: <https://www.geeksforgeeks.org/what-is-local-host/>

```
bulusu — ping localhost — 80x24
(base) bulusu@SEAS-RLLJDGPPWQ ~ % ping localhost
PING localhost (127.0.0.1): 56 data bytes
64 bytes from 127.0.0.1: icmp_seq=0 ttl=64 time=0.158 ms
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.273 ms
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.080 ms
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.260 ms
64 bytes from 127.0.0.1: icmp_seq=4 ttl=64 time=0.262 ms
64 bytes from 127.0.0.1: icmp_seq=5 ttl=64 time=0.243 ms
64 bytes from 127.0.0.1: icmp_seq=6 ttl=64 time=0.247 ms
```

On your web browser:

<http://localhost>

or

<http://127.0.0.1>

When you access 127.0.0.1, you're telling your machine to communicate internally rather than looking for an external IP address.



What is a port in network communication ?

Definition:

A **port** is a communication endpoint used by software applications to send and receive data over a network.

- Ports serve as endpoints for communication, allowing multiple network applications to coexist on a single device.
- Ports are identified by numbers, ranging from 0 to 65535.

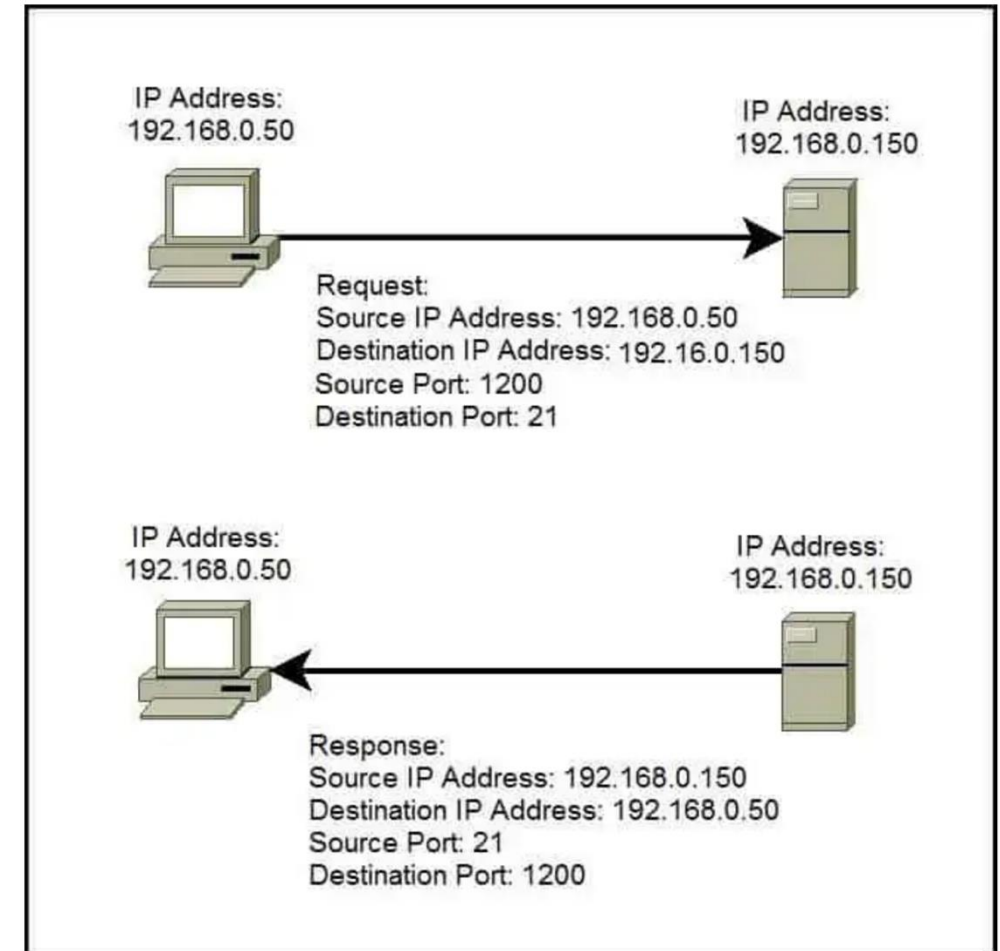
Usage:

Different applications use different ports.

- For example, web servers typically use **port 80** for **HTTP** traffic and **port 443** for **HTTPS**.
- In the case of **127.0.0.1:62893**, the number **62893** is likely dynamically assigned by the operating system for a temporary service or local development task.

Sources:

Medium: <https://medium.com/devgorilla/exploring-localhost-and-127-0-0-1-what-is-the-difference-c109f9ce7f29>
localhost: <https://todaybestreports.com/127-0-0-1-62893-localhost-port-usage-explain/>
IP address: <https://www.geeksforgeeks.org/what-is-local-host/>
<https://study-ccna.com/ports-explained/>



When IoT Teams Expose Localhost

TYPICAL SITUATIONS, WHY IT MATTERS, AND THE MAIN RISK CONTROLS

What it means

Exposing localhost means giving a service running only on a developer laptop, lab PC, or gateway a public or remotely reachable endpoint so devices, cloud services, or teammates can connect to it.

NEED

Reachability

A field device or cloud callback must hit your local service.

TYPICAL PATH

Tunnel / Reverse proxy

Traffic goes to a public URL and forwards into localhost.

REASON

Faster iteration

You test without deploying every code change to staging.



When IoT Teams Expose Localhost

TYPICAL SITUATIONS, WHY IT MATTERS, AND THE MAIN RISK CONTROLS

Most common IoT use cases



Cloud callbacks and webhooks

An IoT platform, message broker, or event service needs to call your local receiver during development.



Remote device debugging

A device in the field must send telemetry or diagnostics to a developer machine without physical access.



Configuration and firmware tests

You temporarily route device traffic to a local service that simulates production logic while tuning settings.



Remote monitoring dashboards

A lab dashboard or admin panel on localhost must be reachable from another network for ops or QA review.



When IoT Teams Expose Localhost

TYPICAL SITUATIONS, WHY IT MATTERS, AND THE MAIN RISK CONTROLS

Security checks before exposing



Authenticate

Protect the endpoint with auth, random URLs, or short-lived credentials instead of leaving it open.



Keep it temporary

Use localhost exposure for development, diagnostics, or support windows, then turn it off.



Minimize surface area

Expose only the exact port and route the device needs, and log every request.

Best fit: testing, troubleshooting, and controlled support sessions — not long-term production exposure.

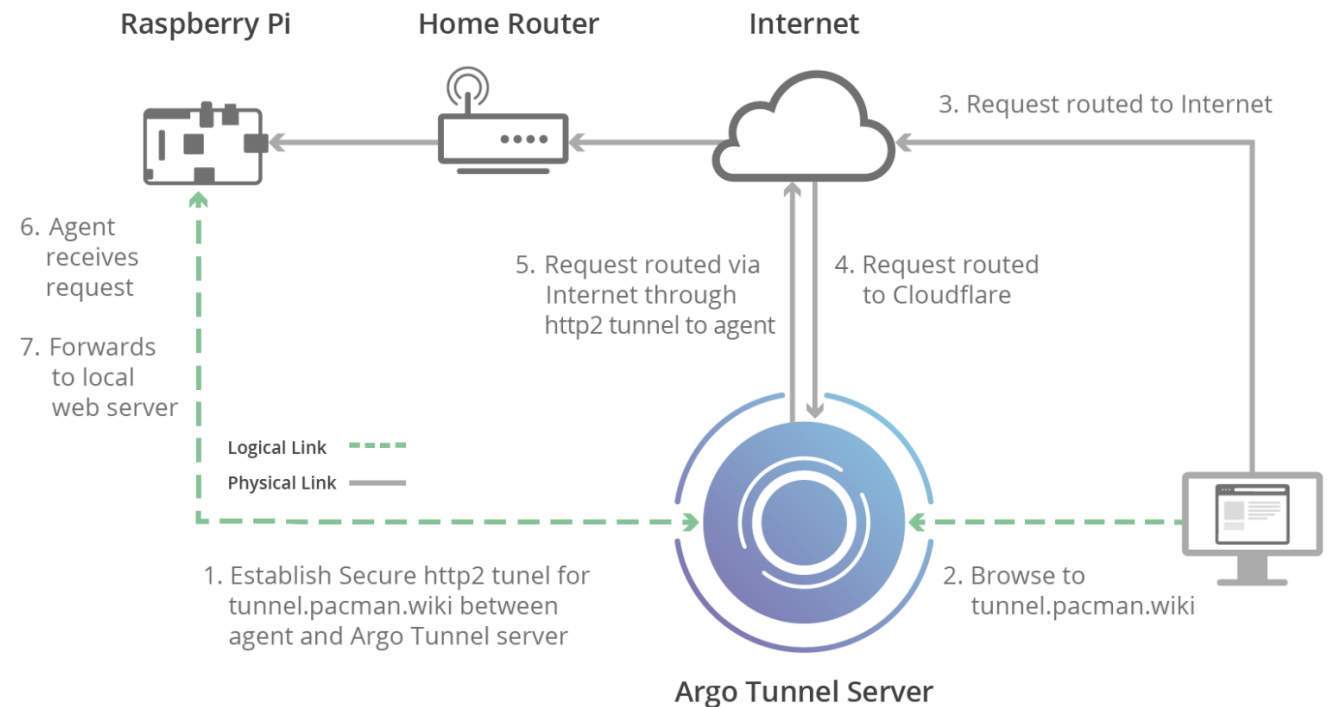


Expose the localhost by creating a tunnel: Cloudflare

Localhost tunnels

A localhost tunnel establishes a connection between your local machine and a remote connection.

- The connection is intended to proxy traffic from a publicly-addressable IP address and URL to your local machine.
- Localhost tunnels are most useful for allowing a tester to connect to a server running on your local development system so they can try out an in-development application you are building but have not yet deployed.



Sources:

<https://www.fullstackpython.com/localhost-tunnels.html>
<https://dev.to/omarccloud20/a-free-cloudflare-tunnel-running-on-a-raspberry-pi-1jld>
<https://blog.cloudflare.com/cloudflare-argo-tunnel-with-rust-and-raspberry-pi/>



Sources:

<https://plotly.com>

API by Vectors Point from <https://thenounproject.com/browse/icons/term/api/>

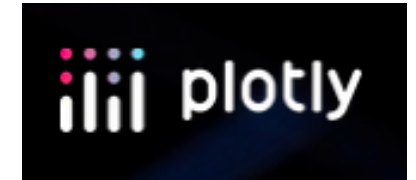
json by ME from <https://thenounproject.com/browse/icons/term/json/>

javascript file by SAM Designs from <https://thenounproject.com/browse/icons/term/javascript-file/>

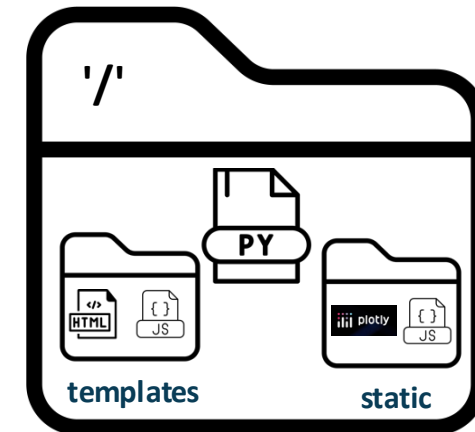
html by tezar tantular from <https://thenounproject.com/browse/icons/term/html/>

Folder by Colourcreatype from <https://thenounproject.com/browse/icons/term/folder/>

In class graded exercise on April 15, 2026



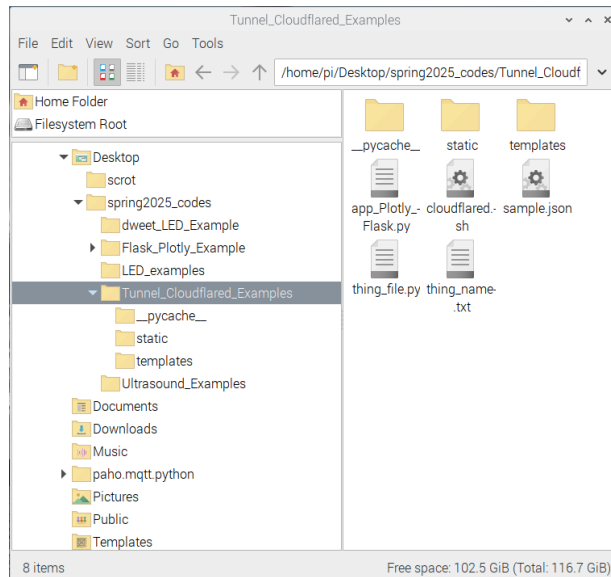
1. Live stream data from your Raspberry Pi 4B
2. Provide your unique *thing_name.txt* associated with your IoT device



Create a localhost webserver using Flask

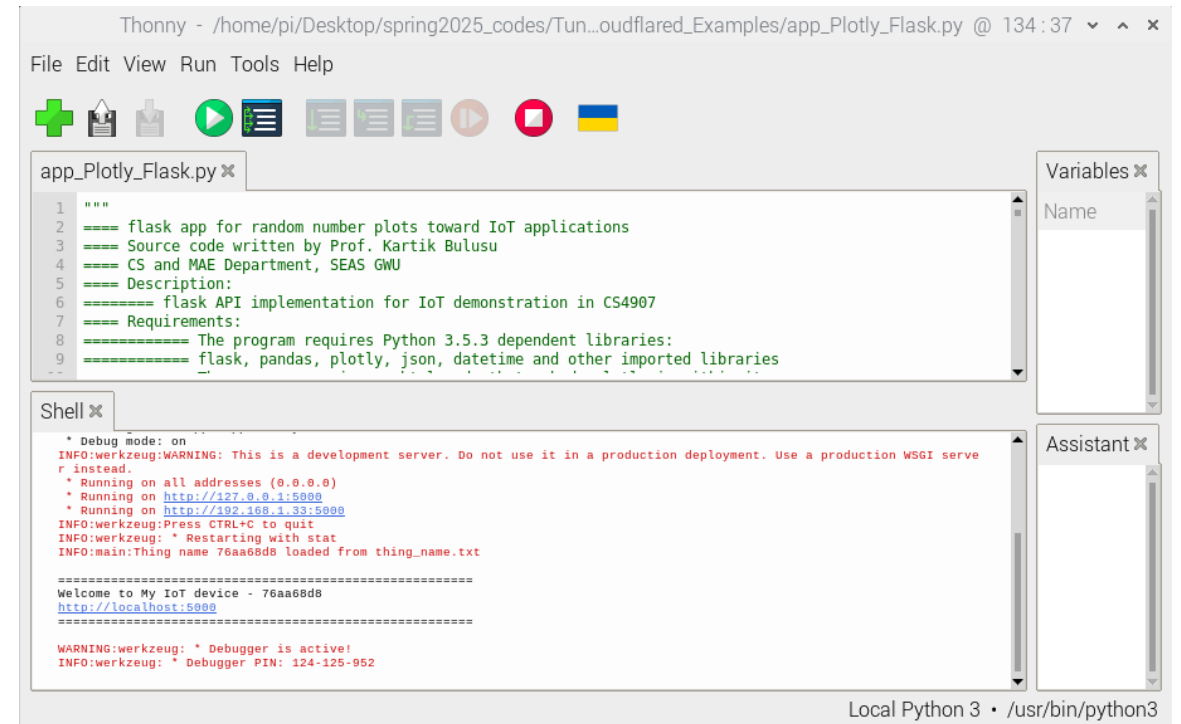
Step-0: git clone https://github.com/gwu-mae6291-iot/spring2026_codes.git

Step-1: Navigate to [Tunnel_Cloudflare_Examples](#) folder and run the python program [app_Plotly_Flask.py](#)



Step-2:

Click on <http://localhost:5000> to observe your data

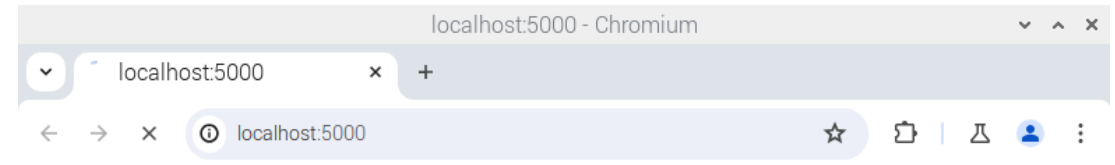


Your output on http://localhost:5000

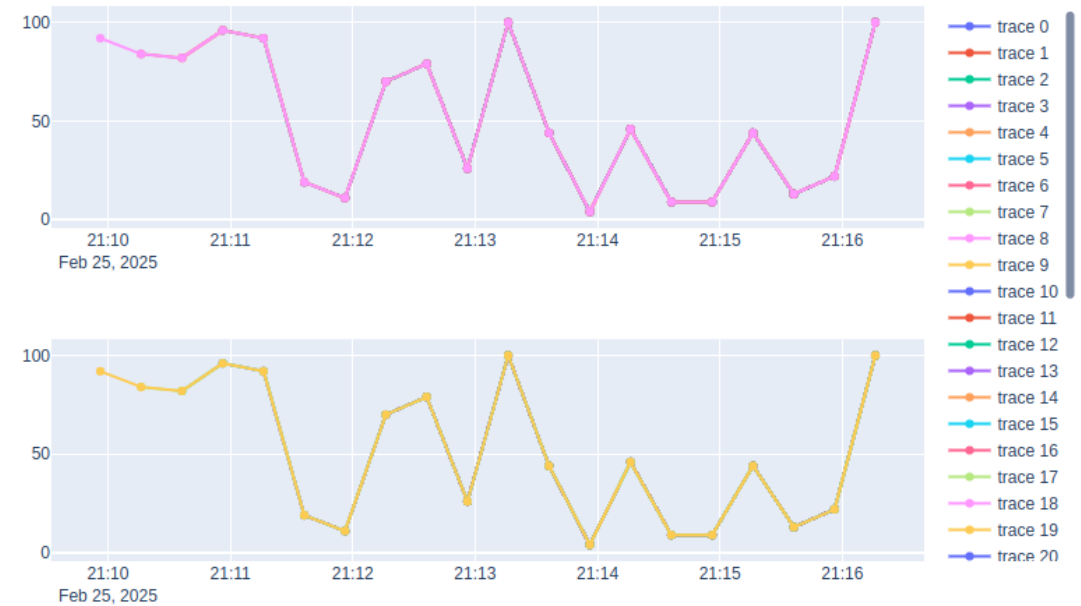
From Step-2:

The data you should see will look like the image presented here.

- Can you identify the port used in this python program ?
- Is it a default port ?
- Can you change it ?



Prof. Bulusu's data from sensors



[OPTIONAL] Install Cloudflare client on your Raspberry Pi

Step-3:

Make sure you are connected to a network (GWdevice or other)

Step-4:

Run the bash-script `cloudflared.sh` provided to you
`./cloudflared.sh`

```
pi@raspberrypi017: ~/Desktop/spring2025_codes/Tunnel_Cloudflared_Examples
File Edit Tabs Help
pi@raspberrypi017:~ $ cd Desktop/spring2025_codes/Tunnel_Cloudflared_Examples/
pi@raspberrypi017:~/Desktop/spring2025_codes/Tunnel_Cloudflared_Examples $ ls
app_Plotly_Flask.py __pycache__ static thing_file.py
pi@raspberrypi017:~/Desktop/spring2025_codes/Tunnel_Cloudflared_Examples $ ./cloudflared.sh
deb [signed-by=/usr/share/keyrings/cloudflare-main.gpg] https://pkg.cloudflare.com/cloudflare
d bullseye main
Hit:1 http://raspbian.raspberrypi.org/raspbian bullseye InRelease
Hit:2 http://archive.raspberrypi.org/debian bullseye InRelease
Get:3 https://download.docker.com/linux/raspbian bullseye InRelease [26.6 kB]
Hit:4 https://pkg.cloudflare.com/cloudflared bullseye InRelease
Fetched 26.6 kB in 1s (20.1 kB/s)
Reading package lists... Done
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
cloudflared is already the newest version (2025.2.0).
The following package was automatically installed and is no longer required:
  libfuse2
Use 'sudo apt autoremove' to remove it.
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
pi@raspberrypi017:~/Desktop/spring2025_codes/Tunnel_Cloudflared_Examples $
```



Check Cloudflare client version and Run the Flask Program

Step-5:

Check **cloudflare** version on your Raspberry Pi

`cloudflared --version`

Step-6:

Run `app_Plotly_Flask.py`

The data will be displayed as seen in Step-2 on <http://localhost:5000>

```
pi@raspberrypi017: ~/Desktop/spring2025_codes/Tunnel_Cloudflared_Examples
File Edit Tabs Help
pi@raspberrypi017:~ $ cd Desktop/spring2025_codes/Tunnel_Cloudflared_Examples/
pi@raspberrypi017:~/Desktop/spring2025_codes/Tunnel_Cloudflared_Examples $
pi@raspberrypi017:~/Desktop/spring2025_codes/Tunnel_Cloudflared_Examples $
pi@raspberrypi017:~/Desktop/spring2025_codes/Tunnel_Cloudflared_Examples $ cloudflared --version
cloudflared version 2025.2.0 (built 2025-02-05-1043 UTC)
pi@raspberrypi017:~/Desktop/spring2025_codes/Tunnel_Cloudflared_Examples $
pi@raspberrypi017:~/Desktop/spring2025_codes/Tunnel_Cloudflared_Examples $
pi@raspberrypi017:~/Desktop/spring2025_codes/Tunnel_Cloudflared_Examples $ python app_Plotly_Flask.py
INFO:main:Thing name 76aa68d8 loaded from thing_name.txt

=====
Welcome to My IoT device - 76aa68d8
http://localhost:5000
=====

* Serving Flask app 'app_Plotly_Flask'
* Debug mode: on
INFO:werkzeug:WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://10.198.24.23:5000
INFO:werkzeug:Press CTRL+C to quit
INFO:werkzeug: * Restarting with stat
```



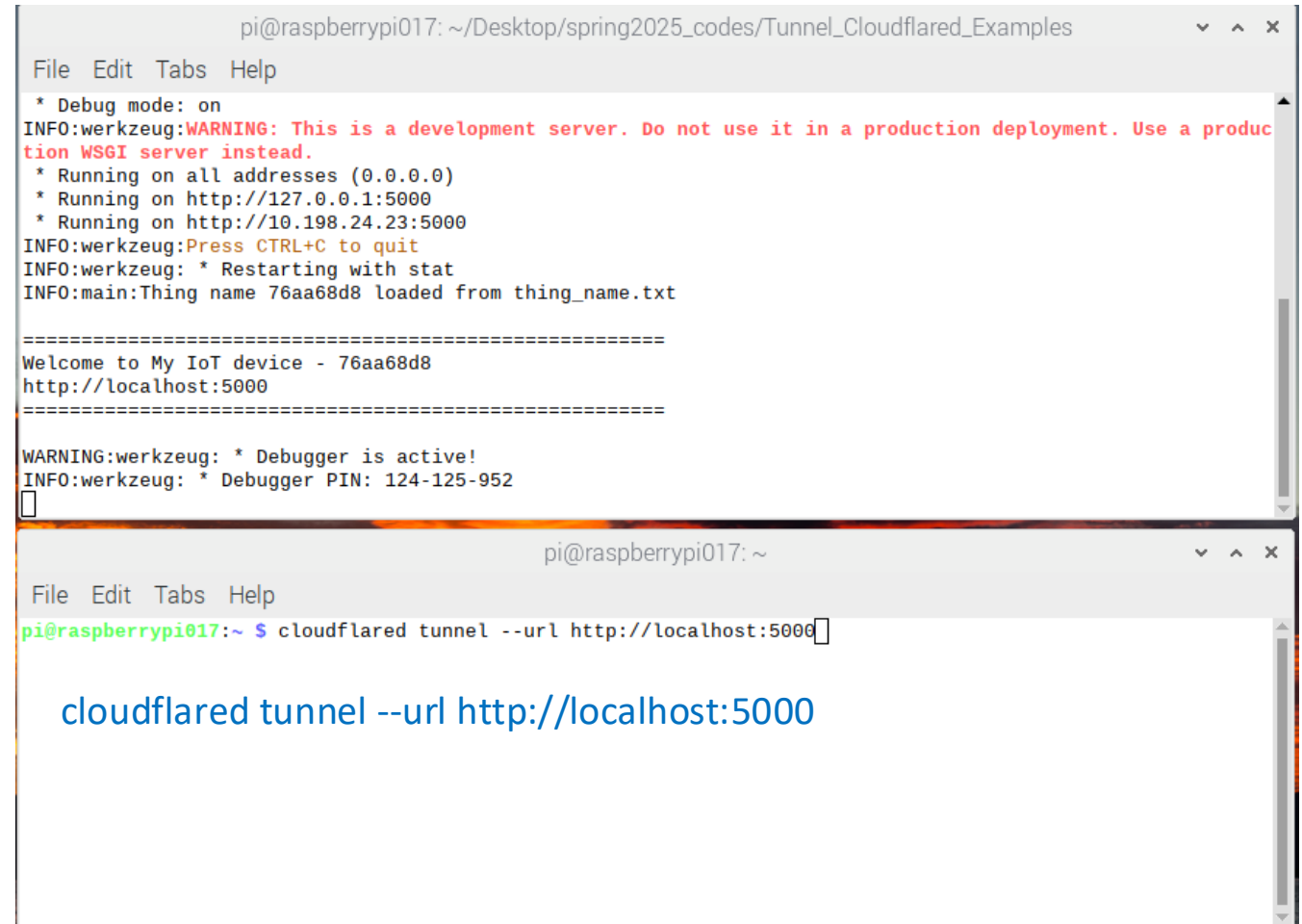
Create a localhost Cloudflare tunnel

Step-7:

Open a new **Terminal**

Step-8:

Create a **cloudflared tunnel** as shown



The image shows two terminal windows. The top window is titled 'pi@raspberrypi017: ~/Desktop/spring2025_codes/Tunnel_Cloudflared_Examples' and displays the output of a Python script. It shows debug mode is on, a warning about using a development server, and that the application is running on http://127.0.0.1:5000 and http://10.198.24.23:5000. It also shows a restart command and a welcome message for the IoT device. The bottom window is titled 'pi@raspberrypi017: ~' and shows the command 'cloudflared tunnel --url http://localhost:5000' being entered at the prompt.

```
pi@raspberrypi017: ~/Desktop/spring2025_codes/Tunnel_Cloudflared_Examples
File Edit Tabs Help
* Debug mode: on
INFO:werkzeug:WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://10.198.24.23:5000
INFO:werkzeug:Press CTRL+C to quit
INFO:werkzeug: * Restarting with stat
INFO:main:Thing name 76aa68d8 loaded from thing_name.txt

=====
Welcome to My IoT device - 76aa68d8
http://localhost:5000
=====

WARNING:werkzeug: * Debugger is active!
INFO:werkzeug: * Debugger PIN: 124-125-952
[]

pi@raspberrypi017: ~
File Edit Tabs Help
pi@raspberrypi017:~ $ cloudflared tunnel --url http://localhost:5000[]

cloudflared tunnel --url http://localhost:5000
```



You successfully exposed your localhost using a tunnel

Step-8:

Look for the funny looking URL on your terminal

Done:

Your **localhost** is now live using a **cloudflared** tunnel.

Type in your URL on your phones or other media to see the data from the **localhost**.

```
pi@raspberrypi017: ~
File Edit Tabs Help

a quick way to experiment and try it out. However, be aware that these account-less Tunnels have no uptime
guarantee, are subject to the Cloudflare Online Services Terms of Use (https://www.cloudflare.com/website-
terms/), and Cloudflare reserves the right to investigate your use of Tunnels for violations of such terms.
If you intend to use Tunnels in production you should use a pre-created named tunnel by following: https://
developers.cloudflare.com/cloudflare-one/connections/connect-apps
2025-02-26T03:32:47Z INF Requesting new quick Tunnel on trycloudflare.com...
2025-02-26T03:32:51Z INF +-----+
-----+
2025-02-26T03:32:51Z INF | Your quick Tunnel has been created! Visit it at (it may take some time to be re
achable): |
2025-02-26T03:32:51Z INF | https://whatever-means-editions-commodities.trycloudflare.com
|
2025-02-26T03:32:51Z INF +-----+
-----+
2025-02-26T03:32:51Z INF Cannot determine default configuration path. No file [config.yml config.yaml] in [
~/.cloudflared ~/.cloudflare-warp ~/cloudflare-warp /etc/cloudflared /usr/local/etc/cloudflared]
2025-02-26T03:32:51Z INF Version 2025.2.0 (Checksum 842eba7f128c74c6db920eb71e4e7116d08c6bdc8ba00e169f56bb5
a50c6f4ca)
2025-02-26T03:32:51Z INF G00S: linux, G0Version: go1.22.10, GoArch: arm
2025-02-26T03:32:51Z INF Settings: map[ha-connections:1 protocol:quic url:http://localhost:5000]
2025-02-26T03:32:51Z INF Generated Connector ID: c2a21eaf-ddaa-425f-a8a3-ef3204ccd66d
2025-02-26T03:32:51Z INF cloudflared will not automatically update if installed by a package manager.
2025-02-26T03:32:51Z INF Initial protocol quic
2025-02-26T03:32:51Z INF ICMP proxy will use 10.198.24.23 as source for IPv4
2025-02-26T03:32:51Z INF ICMP proxy will use fe80::3d8a:ccb:6a3a:2f0 in zone wlan0 as source for IPv6
2025-02-26T03:32:51Z INF ICMP proxy will use 10.198.24.23 as source for IPv4
2025-02-26T03:32:51Z INF ICMP proxy will use fe80::3d8a:ccb:6a3a:2f0 in zone wlan0 as source for IPv6
2025-02-26T03:32:51Z INF Starting metrics server on 127.0.0.1:20241/metrics
2025-02-26T03:32:51Z INF Using [CurveID(4588) CurveID(25497) CurveP256] as curve preferences connIndex=0 ev
ent=0 ip=198.41.200.63
2025/02/25 22:32:51 failed to sufficiently increase receive buffer size (was: 208 kiB, wanted: 7168 kiB, go
t: 416 kiB). See https://github.com/quic-go/quic-go/wiki/UDP-Buffer-Sizes for details.
2025-02-26T03:32:51Z INF Registered tunnel connection connIndex=0 connection=5ee58686-321d-4fe0-92ea-22c848
212cfe event=0 ip=198.41.200.63 location=iad11 protocol=quic
```



Someone should summarize what we learned today

